МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Інститут **ІКНІ**
Кафедра **ПЗ**

ЗВІТ

До лабораторної роботи № 3
На тему: *"Робота з масивами в мові С"*
З дисципліни: *"Основи програмування"*

Лектор:
ст.викл. каф. ПЗ
Муха Т.О.

Виконав:
ст. гр. ПЗ-17
Кашпірук І. А.

Прийняла:
асист. каф. ПЗ
Заводовська Н.О.

« ____ » _____ 2026 р.

Σ= ____ .

Львів – 2026

Тема роботи: Робота з масивами в мові С
Мета роботи: навчитися організовувати такі структури даних як масиви та освоїти основні методи програмування алгоритмів обробки масивів даних засобами мови С

Індивідуальне завдання

1. З клавіатури ввести вектор дійсних чисел. Знайти середнє арифметичне усіх чисел, що розміщені між максимальним та мінімальним елементами вектора.
2. Ввести прямокутну матрицю дійсних чисел. Циклічно зсунути її рядки на 1 униз. Циклічний зсув передбачає, що елементи останнього рядка матриці заносяться на відповідні місця зсунутих елементів першого рядка.

Теоретичні відомості

1. Одновимірні масиви та вказівники

Масив — це послідовність елементів одного типу, розташованих в пам'яті один за одним. Ім'я масиву є вказівником-константою, який зберігає адресу першого елемента (&arr[0]).

2. Розіменування та арифметика вказівників

Розіменування (*) дозволяє отримати або змінити значення, яке знаходиться за адресою вказівника.

Оскільки елементи масиву розташовані підряд, вираз (arr + i) обчислює адресу *i*-го елемента

Приклад коду:

```
int arr[4] = {10, 20, 30, 40};

int *ptr = arr; // ptr вказує на arr[0]

int a = arr[2]; // Звичайний індексний доступ (значення 30)

 int b = *(arr + 2); // Розіменування зі зсувом від імені масиву (значення 30)

int c = *(ptr + 2); // Розіменування через окремий вказівник (значення 30)
```

3. Двовимірні масиви(матриці)

Двовимірний масив — це масив масивів (таблиця з рядків та стовпчиків).

```
int matrix[3][4] = {
        {10, 11, 12, 13}, // Рядок 0
        {20, 21, 22, 23}, // Рядок 1
         {30, 31, 32, 33} // Рядок 2 };
```

Розташування двовимірного масиву в пам’яті

У мові С оперативна пам'ять є строго одновимірною. Двовимірний масив зберігається в пам'яті суцільним блоком за рядками (Row-major order): спочатку повністю перший рядок, потім другий і так далі.

4. **Адресна арифметика та розіменування у двовимірних масивах**

Робота з адресами у двовимірних масивах має кілька рівнів розіменування:

- matrix — вказує на початок всього двовимірного масиву (на перший рядок). Зсув matrix + i зміщує вказівник на цілий *i*-й рядок (крок = розмір рядка в байтах).
- *(matrix + i) — розіменовує вказівник на рядок і перетворює його на вказівник на перший елемент *i*-го рядка (елемент [i][0]).
- *(matrix + i) + j — додає зсув *j* стовпчиків до адреси *i*-го рядка. Це адреса елемента matrix[i][j]
- *(*(matrix + i) + j) — подвійне розіменування, яке вертає значення елемента matrix[i][j].

Додаток 1

Код програми

```
#include <stdio.h>

#define SIZE 10


int main(void){
   double vect[SIZE];
   int maxIndx = 0;
   int minIndx = 0;
   int startIndx, endIndx;
   double sum, average, count;


   for(int i = 0; i < SIZE; i++){
      printf("Enter a value: ");
```

```
    scanf("%lf", vect + i);


    if(*(vect + maxIndx) < *(vect + i)){
        maxIndx = i;
    }
    if(*(vect + minIndx) > *(vect + i)){
        minIndx = i;
    }
}
if(maxIndx > minIndx){
    startIndx = minIndx + 1;
    endIndx = maxIndx;
}else{
    startIndx = maxIndx + 1;
    endIndx = minIndx;
}

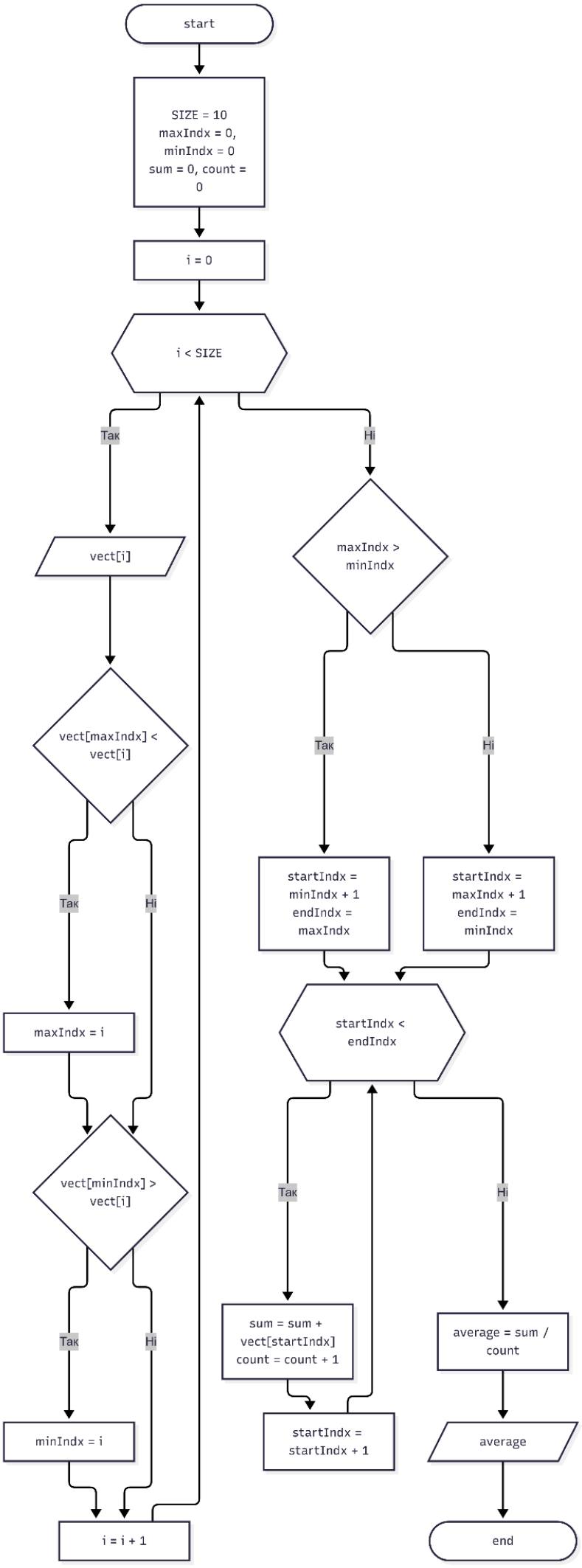

for(startIndx;startIndx < endIndx; startIndx++){
    sum += vect[startIndx];
    count++;
}
average = sum / count;
printf("%lf", average);
```

```
    return 0;
}
```

Протокол програми

```
Enter a value: 1
Enter a value: 2
Enter a value: 3
Enter a value: 4
Enter a value: 5
Enter a value: 6
Enter a value: 7
Enter a value: 8
Enter a value: 9
Enter a value: 10
5.500000
```

Блок схема


start
SIZE = 10
maxIndx = 0,
minIndx = 0
sum = 0, count =
0
i = 0
i < SIZE
Так
Ні
vect[i]
vect[maxIndx] <
vect[i]
Так
Ні
maxIndx = i
vect[minIndx] >
vect[i]
Так
Ні
minIndx = i
i = i + 1
maxIndx >
minIndx
Так
Ні
startIndx =
minIndx + 1
endIndx =
maxIndx
startIndx =
maxIndx + 1
endIndx =
minIndx
startIndx <
endIndx
Так
Ні
sum = sum +
vect[startIndx]
count = count + 1
startIndx =
startIndx + 1
average = sum /
count
average
end

Додаток 2

Код програми

```
#include <stdio.h>
#define SIZE 4

int main(){
    double matrix[SIZE][SIZE];
    double tempArr[SIZE];

    for(int i = 0; i < SIZE; i++){
        for(int j = 0; j < SIZE; j++){
            printf("Enter a value: ");
            scanf("%lf", *(matrix + i) + j);

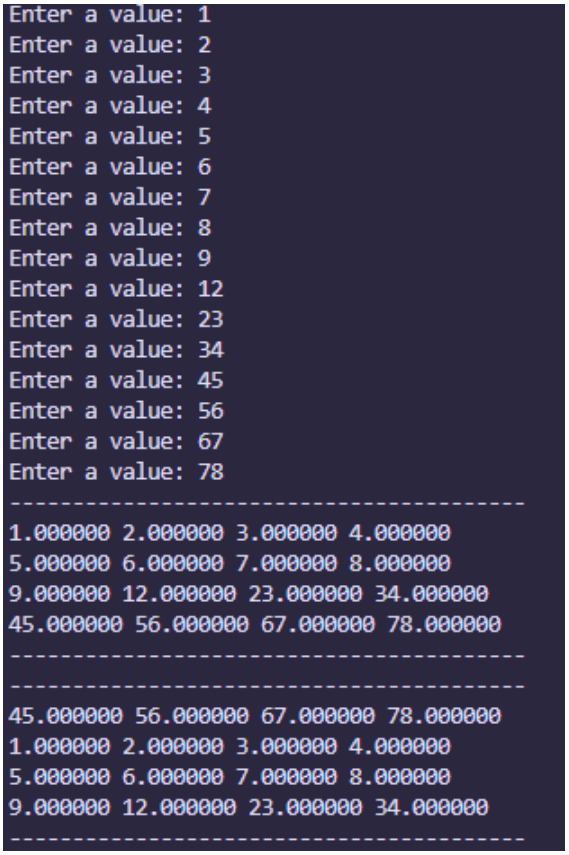
            if(i == SIZE - 1){
                *(tempArr + j) = (*(matrix + i))[j];
            }
        }
    }
    printf("----------------------------------------\n");
    for(int i = 0; i < SIZE; i++){
```

```
    for(int j = 0; j < SIZE; j++){
            if(j == SIZE - 1){
                printf("%f\n", *(*(matrix + i) + j));
            }else{
                printf("%f ", *(*(matrix + i) + j));
            }
    }
}
printf("----------------------------------------\n");
for(int i = SIZE - 1; i >= 0; i--){
    for(int j = 0; j < SIZE; j++){
        if(i == 0){
           *(*(matrix + i) + j) = *(tempArr + j);
        }else{
            *(*(matrix + i) + j) = *(*(matrix + (i - 1)) + j);
        }

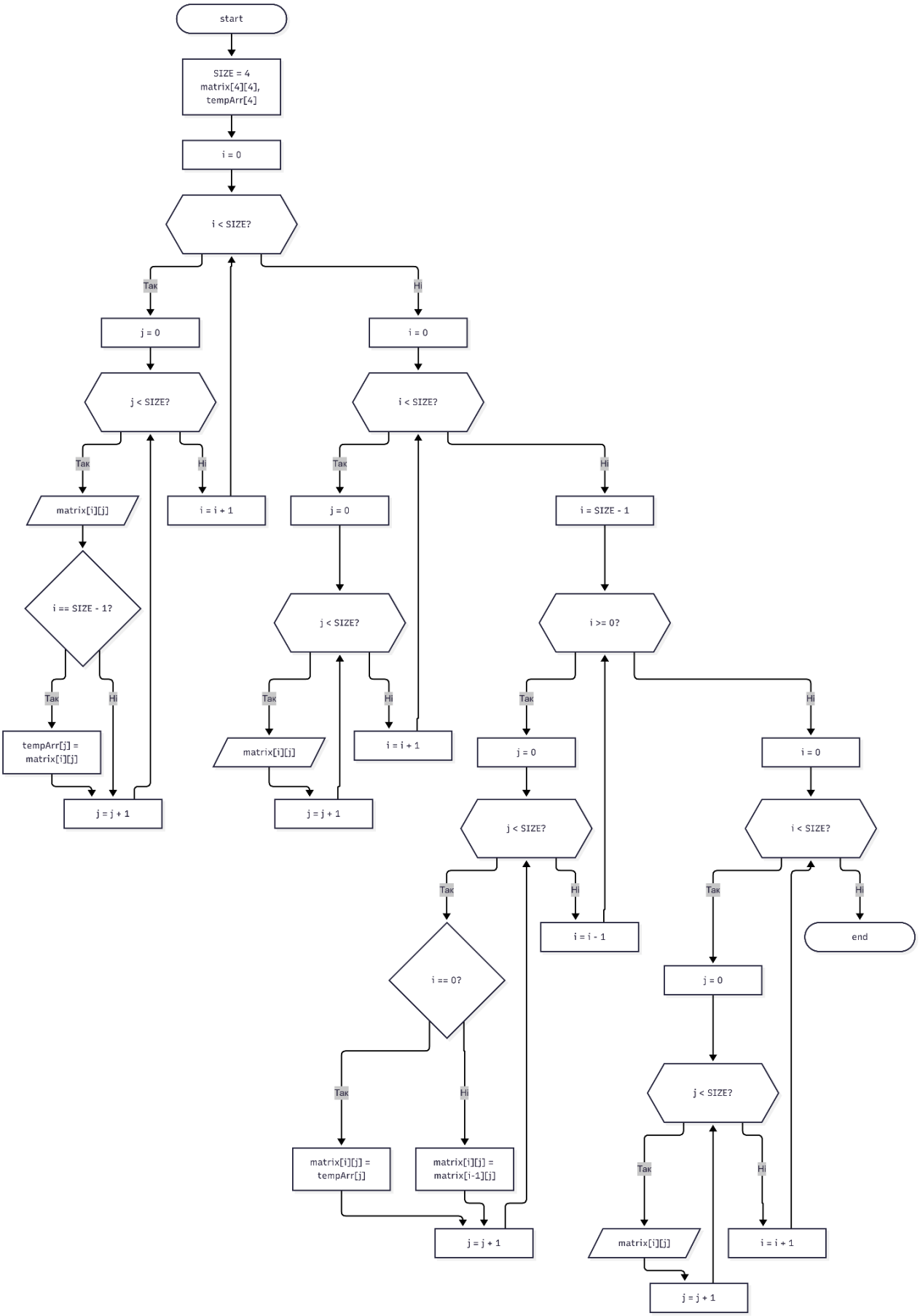
    }
}
printf("----------------------------------------\n");
 for(int i = 0; i < SIZE; i++){
    for(int j = 0; j < SIZE; j++){
            if(j == SIZE - 1){
                printf("%f\n", *(*(matrix + i) + j));
```

```
            }else{
                printf("%f ", *(*(matrix + i) + j));
            }
        }
    }
    printf("----------------------------------------\n");
    return 0;
}
```

Протокол програми



Блок схема


start
SIZE = 4
matrix[4][4],
tempArr[4]
i = 0
i < SIZE?
Так
Ні
j = 0
j < SIZE?
Так
Ні
matrix[i][j]
i == SIZE - 1?
Так
Ні
tempArr[j] =
matrix[i][j]
j = j + 1
i = i + 1
i = 0
i < SIZE?
Так
Ні
j = 0
j < SIZE?
Так
Ні
matrix[i][j]
j = j + 1
i = i + 1
i = SIZE - 1
i >= 0?
Так
Ні
j = 0
j < SIZE?
Так
Ні
i == 0?
Так
Ні
matrix[i][j] =
tempArr[j]
matrix[i][j] =
matrix[i-1][j]
j = j + 1
i = i - 1
i = 0
i < SIZE?
Так
Ні
end
j = 0
j < SIZE?
Так
Ні
matrix[i][j]
i = i + 1
j = j + 1

Висновки

Під час виконання лабораторної роботи було досліджено особливості збереження одновимірних і двовимірних масивів у пам'яті та закріплено навички застосування адресної арифметики й розіменування вказівників для доступу до їхніх елементів без використання стандартного індексного синтаксису.